

J119811 (REPORT)

by MD TANVIR SARDARR

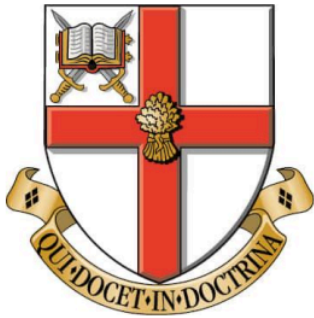
Submission date: 29-Apr-2025 05:06AM (UTC+0100)

Submission ID: 257053896

File name: J119811__REPORT__1199318_665674592.docx (2.06M)

Word count: 3815

Character count: 27798



University of Chester

Analysing & Predicting Sales Trends

Student Assessment Number: J119811

Module Code: CO7024

Module Title: Statistical programming

Word count: 2533 (Including headings, Excluding: Cover Page, Table of Contents, References and Appendix)

Date of Submission: 29 April 2025

Table of Contents

1 Background..... 1

2 Data Importation, Inspection, and Merging..... 1

3 Distribution Analysis..... 4

4 Summary Statistics 11

5 Profitability Analysis and Sales Forecasting 14

6 Insights & Recommendations 25

7 Conclusion 28

8 References..... 29

9 Appendix (Code) 30

1 Background

Achieving optimal inventory levels, best pricing strategies, and retail outlet marketing success requires accurate sales predictions and profitability analysis at XYZ Corporation. According to Suresh and Praveen (2020), the development of data science technology makes polynomial regression an applicable learning method for identifying complex sales behaviours without creating overfitting. The Seasonal-Trend decomposition using LOESS (STL) analytical tool delivers critical seasonality insights and underlying pattern detection abilities for time-series data interpretation (Theodosiou, 2011).

Current studies unite deep learning models with external elements such as promotions together with holidays and economic conditions to achieve superior forecasting outcomes (Hao & Liu, 2024; MD AL et al., 2025). The report uses advanced analytical techniques to evaluate profitability and sales metrics at different XYZ Corporation stores. The application of factual data aims to optimize operational efficiency while enabling strategic choices which leads to higher overall business success in the retail operations of XYZ Corporation.

2 Data Importation, Inspection, and Merging

This step explains the process of preparing data, giving a homogeneous and clean dataset that can be used for analysis. Four datasets were originally imported: three stores' daily sales records (Store_A, Store_B, Store_C) and one centralized product price dataset. The datasets were loaded into Python through `pandas.read_csv()` and stored in a dictionary for easy access. To be consistent, column names in all datasets were also changed to lowercase, spaces were dropped, and underscores were applied for formatting.

The sales data frames, initially in wide form with individual product columns, were reshaped into long-form by `pandas.melt()`. This conversion merged product information into two columns: `product` and `units_sold`, with a `store` column added to preserve store identifiers. The reshaped data frames were joined using `pandas.concat()`, which created one data frame with sales records from all stores.

```
# Part 1: Data Importation, Inspection & Merging
paths = {
    'Store_A': 'D:/I/Assignment 2/store_a_sales_data.csv',
    'Store_B': 'D:/I/Assignment 2/store_b_sales_data.csv',
    'Store_C': 'D:/I/Assignment 2/store_c_sales_data.csv',
    'Pricing': 'D:/I/Assignment 2/product_pricing_data.csv'
}

# Load data
dfs = {k: pd.read_csv(v) for k, v in paths.items()}

# Clean names
for df in dfs.values():
    df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_')

# Melt sales
def melt_data(df, label):
    m = df.melt(id_vars=['date', 'store_name'], var_name='product', value_name='units_sold')
    m['store'] = label
    return m

sales = pd.concat([
    melt_data(dfs['Store_A'], 'Store_A'),
    melt_data(dfs['Store_B'], 'Store_B'),
    melt_data(dfs['Store_C'], 'Store_C')
], ignore_index=True)

# Merge pricing
sales['product'] = sales['product'].str.replace('product_', '', regex=False).astype(int)
pricing = dfs['Pricing']
pricing['product'] = pricing['product'].str.replace('Product_', '', regex=False).astype(int)
merged = sales.merge(pricing, on='product', how='left')
merged['date'] = pd.to_datetime(merged['date'])
```

The combined sales data was joined with the pricing dataset on the product column via a left join. This brought cost price and selling price information into the sales data. The date column was then read into datetime format via `pandas.to_datetime()` for time-series analysis.

```
# Data quality
print("Data Quality Checks:")
print(merged.info())
print(merged.describe())
print("Duplicates:", merged.duplicated().sum())
```

Data quality checks were performed, including structure verification using `info()`, summary statistics using `describe()`, and duplication checking using `duplicated().sum()`. This systematic approach confirmed data integrity and consistency, in agreement with best practice in retail analytics (Suresh & Praveen, 2020; Krishna et al., 2018).

```

PS C:\Users\s> & "C:/Program Files/Python312/python.exe" "d:/I/Assignment 2/i.py"
Data Quality Checks:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19710 entries, 0 to 19709
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   date            19710 non-null  datetime64[ns]
 1   store_name      19710 non-null  object
 2   product         19710 non-null  int64
 3   units_sold      19710 non-null  int64
 4   store           19710 non-null  object
 5   cost_price      19710 non-null  float64
 6   selling_price   19710 non-null  float64
dtypes: datetime64[ns](1), float64(2), int64(2), object(2)
memory usage: 1.1+ MB
None

```

	date	product	units_sold	cost_price	selling_price
count	19710	19710.000000	19710.000000	19710.000000	19710.000000
mean	2022-07-02 00:00:00	3.500000	82.050127	24.865000	53.823333
min	2021-01-01 00:00:00	1.000000	0.000000	6.340000	20.940000
25%	2021-10-01 00:00:00	2.000000	34.000000	8.470000	25.160000
50%	2022-07-02 00:00:00	3.500000	64.000000	22.610000	55.805000
75%	2023-04-02 00:00:00	5.000000	114.000000	39.510000	80.680000
max	2023-12-31 00:00:00	6.000000	299.000000	49.650000	84.550000
std	NaN	1.707868	65.529818	17.394952	27.508899
Duplicates:	0				

This data quality check report summarizes the shape and consistency of the merged dataset. Info() usage confirms that the dataset contains 19,710 records in 7 columns: date, store_name, product, units_sold, store, cost_price, and selling_price. All columns have no missing values, and their data types are appropriate for analysis (e.g., datetime64 for dates, float64 for prices, and int64 for numbers).

The describe() summary provides statistical data. For units_sold, the mean is approximately 82, with a wide range of 0 to 299, indicating variation in sales volumes. Both cost_price and selling_price have reasonable ranges, confirming uniform pricing of goods. Additionally, the absence of duplicate records ensures data integrity. This validation step is required for reliable analytics and adheres to best practices in retail data analysis (Suresh & Praveen, 2020).

3 Distribution Analysis

3.1. Methodology

Anderson-Darling Normality Test:

For each product, the Anderson-Darling test checked whether sales were distributed.

```
# Part 2: Distribution Analysis
print("\nDistribution Analysis (Anderson-Darling) & Visuals:")
for p in sorted(merged['product'].unique()):
    vals = merged.loc[merged['product']==p, 'units_sold']
    res = anderson(vals)
    print(f"Product {p}: stat={res.statistic:.2f}, crit@5%={res.critical_values[2]}")
    # Histogram + KDE
    plt.figure(figsize=(6,4))
    sns.histplot(vals, kde=True)
    plt.title(f'Units Sold Distribution - Product {p}')
    plt.tight_layout(); plt.savefig(f'dist_hist_product_{p}.png'); plt.show()
    # Boxplot
    plt.figure(figsize=(6,2))
    sns.boxplot(x=vals)
    plt.title(f'Units Sold Boxplot - Product {p}')
    plt.tight_layout(); plt.savefig(f'dist_box_product_{p}.png'); plt.show()
```

The results give a statistic (res.statistic) and critical value at 5% significance level (res.critical_values[2]). Where the statistic is larger than the critical value, the null hypothesis (normal distribution) is rejected.

3.2 Anderson-Darling Test Results

```
Distribution Analysis (Anderson-Darling) & Visuals:
Product 1: stat=39.42, crit@5%=0.786
Product 2: stat=37.15, crit@5%=0.786
Product 3: stat=37.81, crit@5%=0.786
Product 4: stat=40.97, crit@5%=0.786
Product 5: stat=39.79, crit@5%=0.786
Product 6: stat=42.41, crit@5%=0.786
```

No sales figures of the products exhibited normal distribution because all test statistics significantly exceeded the critical value. This reflects non-normal pattern distribution with heavy tails or skewness, which is natural for retail sales data.

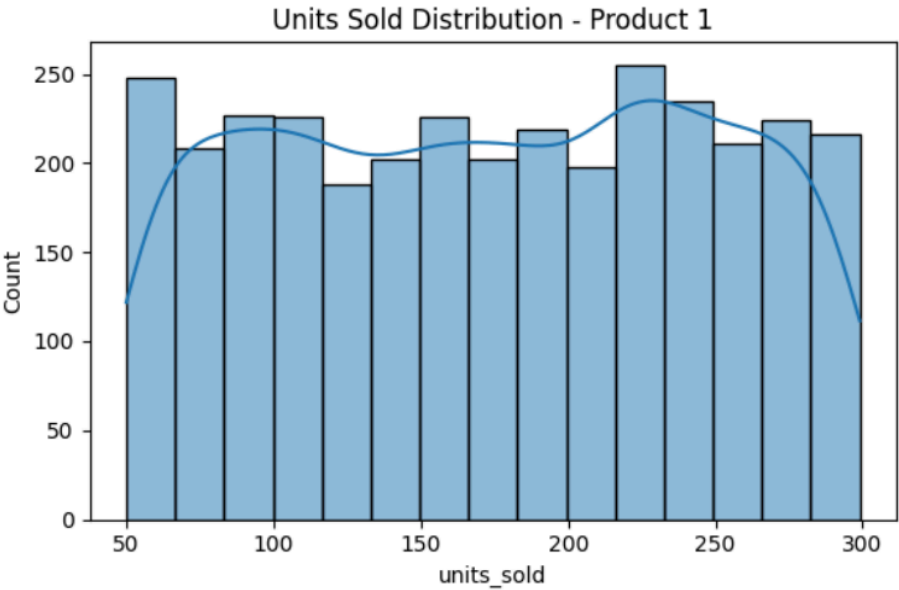
3.3 Visualizations

Histograms with KDE : To graph the frequency distribution of units sold per item and overlay a KDE curve to indicate the shape of the distribution.

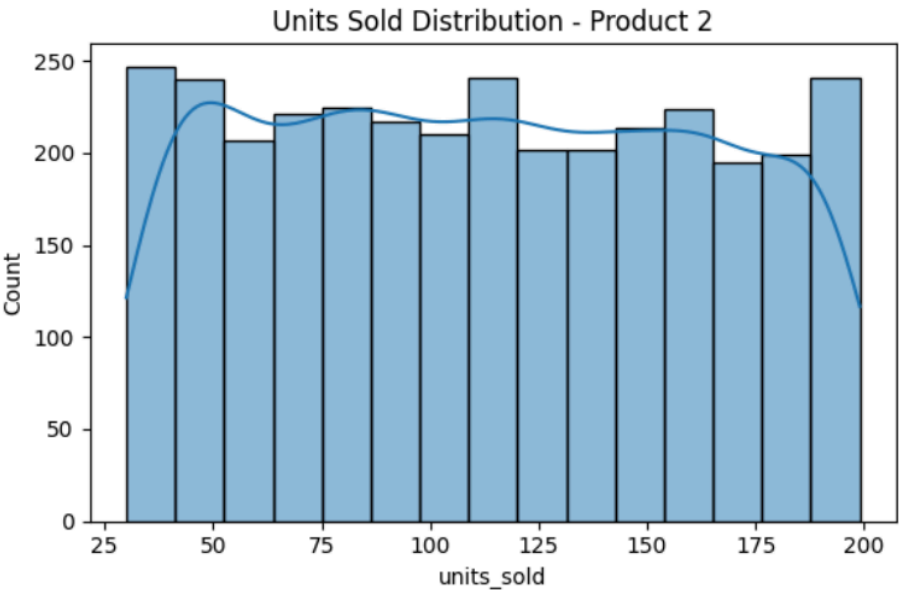
X-axis: Units sold.

Y-axis: Frequency (count of occurrences).

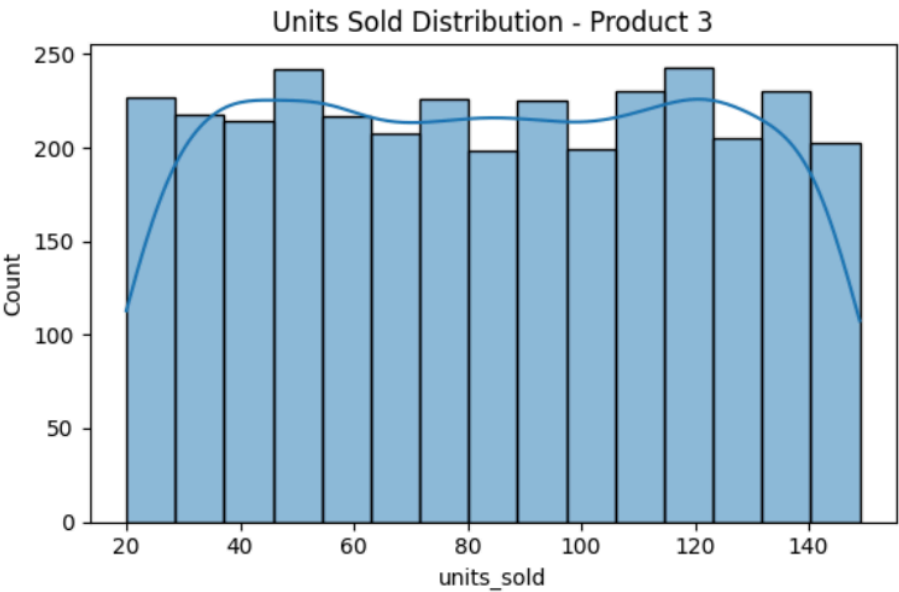
Product 1: Between 50 and 300 units are sold. The KDE curve is rather flat with numerous peaks, exhibiting high sales volatility and suggesting ups and downs in demand during the observation period.



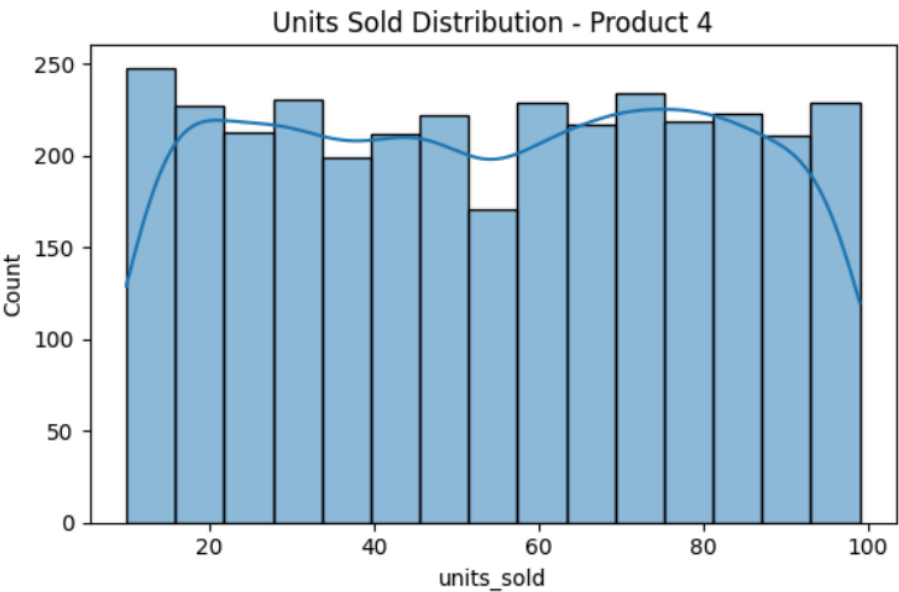
Product 2: Units sold between 25 and 200. The KDE curve is balanced but still inclined slightly, demonstrating stable but erratic sales performance.



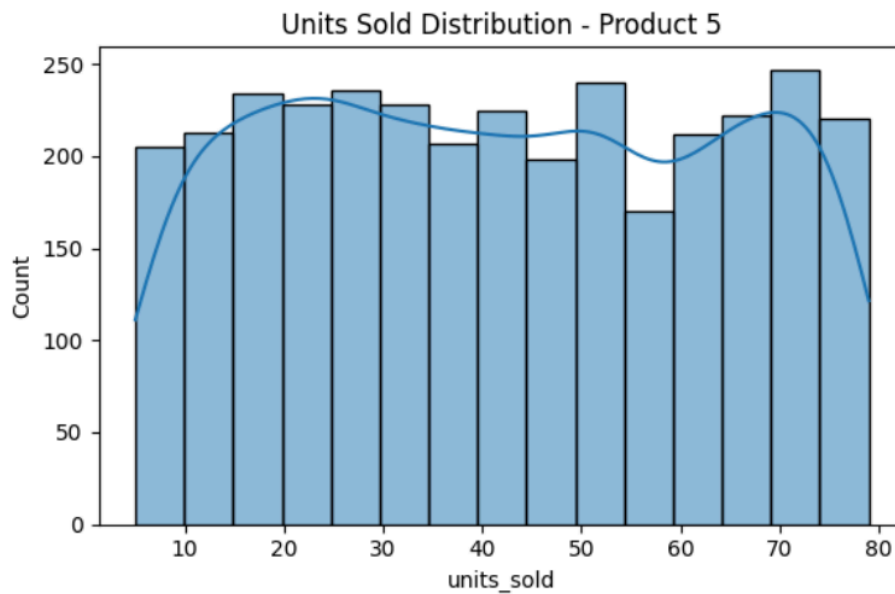
Product 3: Like Product 2 but with a more limited range (20 to 150 units). The KDE curve has peaks that are moderately high, indicating consistent mid-range sales.



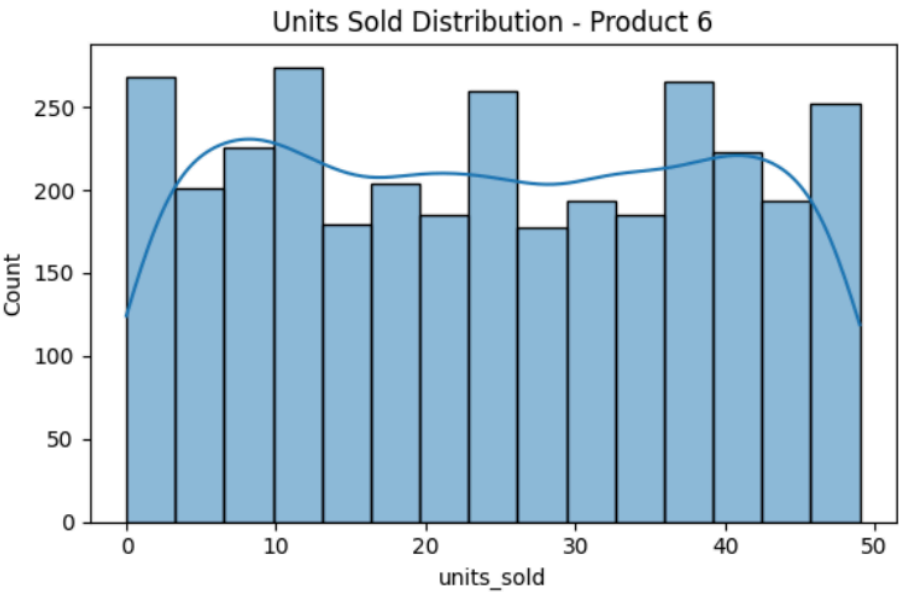
Product 4: Its range is more limited (10 to 100 units), and the KDE curve shows a slight right skew, which means the majority of the sales occur at lower to mid-unit levels.



Product 5: The histogram is more even and goes from 5 to 80 units sold, but the KDE curve shows sporadic peaks in demand.



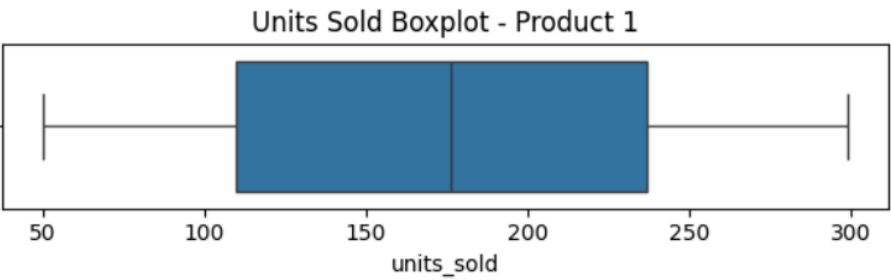
Product 6: This shows the spread with the narrowest width (0 to 50 units), with the KDE curve showing lower, consistent sales volumes.



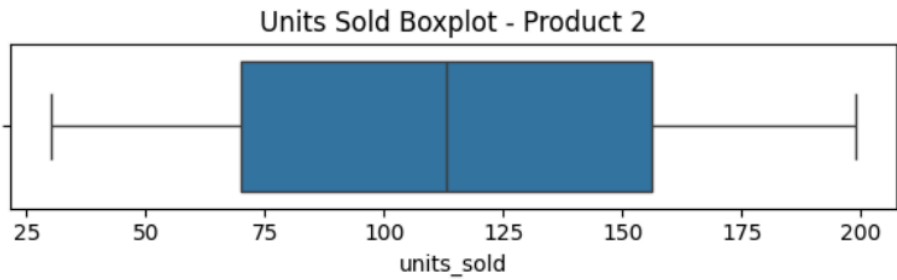
Boxplots : To provide a statistical summary of sales distributions, e.g. medians, interquartile ranges (IQR), and potential outliers.

X-axis: Units sold.

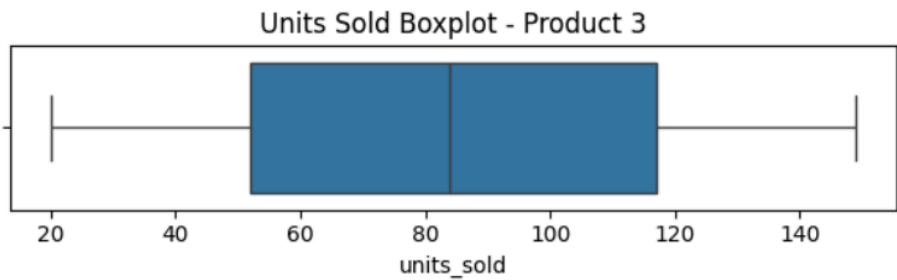
Product 1: Has the broadest IQR (100 to 250 units), with a median around 175 units. The range covers both high demand and variability.



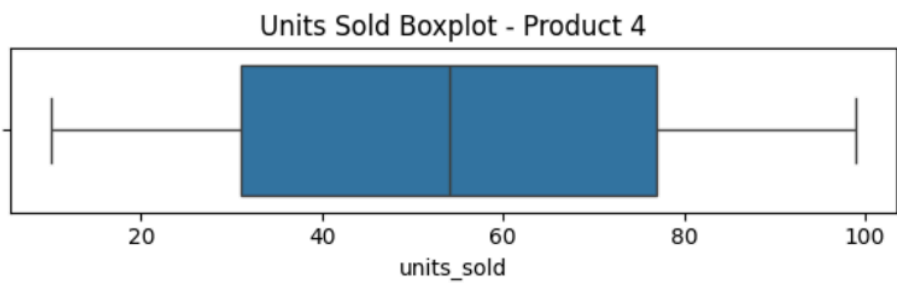
Product 2: IQR of 75 to 150 units, with a slightly lower median, indicating lower variability than Product 1 but still stable sales.



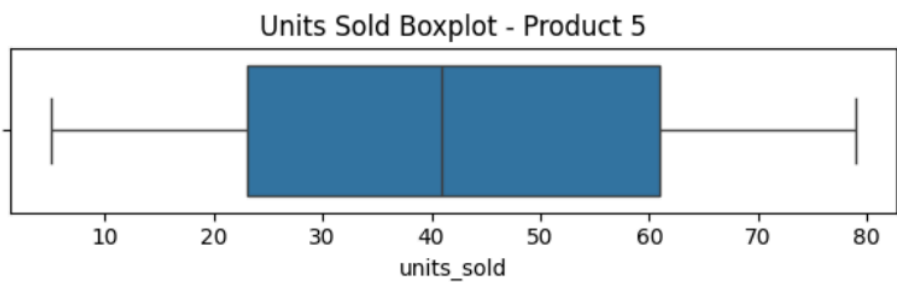
Product 3: Smaller IQR (60 to 120 units), so it is a stable mid-range product.



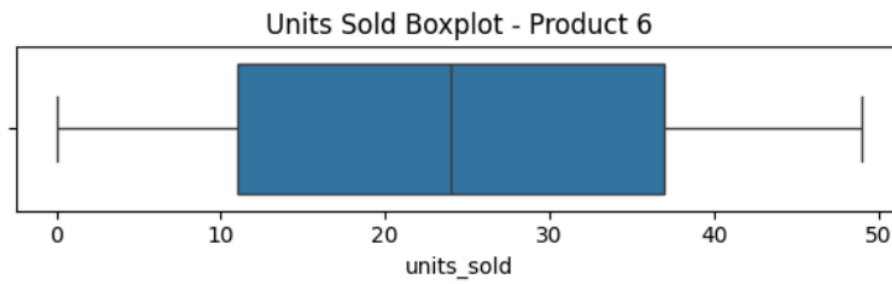
Product 4: Narrower IQR (30 to 80 units), reflecting minimal sales variability.



Product 5: The IQR is still narrower (20 to 60 units), confirming its stable but lower sales volume.



Product 6: Smallest IQR (10 to 40 units), reflecting minimum variability and sustained low sales.



Outliers: No extreme outliers were detected on products, implying uniform sales ranges without isolated spikes.

4 Summary Statistics

4.1. Methodology

Pivot Table: Aggregated total and average units sold by product through each store with `pandas.pivot_table()`.

```
# Part 3: Summary Statistics
print("\nSummary Statistics & Visuals:")
pivot = merged.pivot_table(index='product', columns='store', values='units_sold', aggfunc=['sum', 'mean']).round(2)
print(pivot)
daily = merged.groupby('date')['units_sold'].agg(['sum', 'mean', 'std']).round(2)
print(daily.head())
# Store-level
store_sales = merged.groupby('store')['units_sold'].sum()
store_profit = merged.assign(profit=(merged['selling_price']-merged['cost_price'])*merged['units_sold']).groupby('store')['profit'].sum()
print("Store sales:\n", store_sales)
print("Store profit:\n", store_profit)
# Bar plots
plt.figure(figsize=(6,4)); store_sales.plot(kind='bar'); plt.title('Total Units Sold by Store'); plt.tight_layout(); plt.savefig('bar_store_sales.png'); plt.show()
plt.figure(figsize=(6,4)); store_profit.plot(kind='bar'); plt.title('Total Profit by Store'); plt.tight_layout(); plt.savefig('bar_store_profit.png'); plt.show()
# Most/least profitable
profit_unit = pricing.assign(profit=pricing['selling_price']-pricing['cost_price'])
most_prof = profit_unit.loc[profit_unit['profit'].idxmax(), 'product']
least_prof = profit_unit.loc[profit_unit['profit'].idxmin(), 'product']
print("Most profitable product: {most_prof}, Least profitable product: {least_prof}")
```

Daily Sales Summary: Total sales each day across all stores using `groupby('date')` to compute total, mean, and standard deviation of units sold.

Store-Level Sales & Profitability: Total units sold and total profit by store have been computed using `groupby('store')`. Profit has been calculated as $(\text{selling_price} - \text{cost_price}) * \text{units_sold}$.

4. 2. Results of Summary Statistics

Pivot Table: The pivot table indicates the total units sold and average daily sales for each product in Store_A, Store_B, and Store_C. The sum columns indicate the total units sold over the analysis period, while the mean columns indicate the average daily sales per product.

```
Summary Statistics & Visuals:
sum      mean
store Store_A Store_B Store_C Store_A Store_B Store_C
product
1      193593  189893  189861   176.80   173.42   173.39
2      123677  123422  124330   112.95   112.71   113.54
3       91783   92962   91723    83.82    84.90    83.77
4       59878   59696   58724    54.68    54.52    53.63
5       46836   45038   46040    42.77    41.13    42.05
6       26623   26525   26604    24.31    24.22    24.30

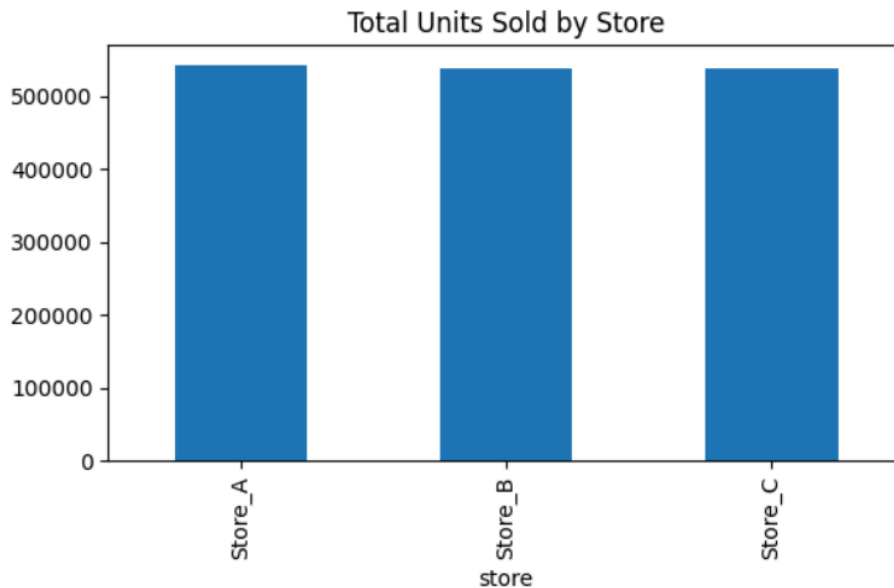
date
sum      mean      std
2021-01-01  1312    72.89   58.75
2021-01-02  1648    91.56   93.91
2021-01-03  1771    98.39   66.30
2021-01-04  1646    91.44   71.33
2021-01-05  1573    87.39   65.00
Store sales:
store
Store_A    542390
Store_B    537536
Store_C    537282
Name: units_sold, dtype: int64
```

Product 1 always has the highest overall sales of all stores (for example, 193,593 units in Store_A), and it is the best-performing product. Product 6 with lower overall sales, for example, shows less market demand.

Sales are day-to-day varying, with some days recording significantly higher volumes than others (e.g., January 2, 2021, recorded a higher total and standard deviation). This volatility suggests external factor-driven demand patterns such as seasonality or promotions.

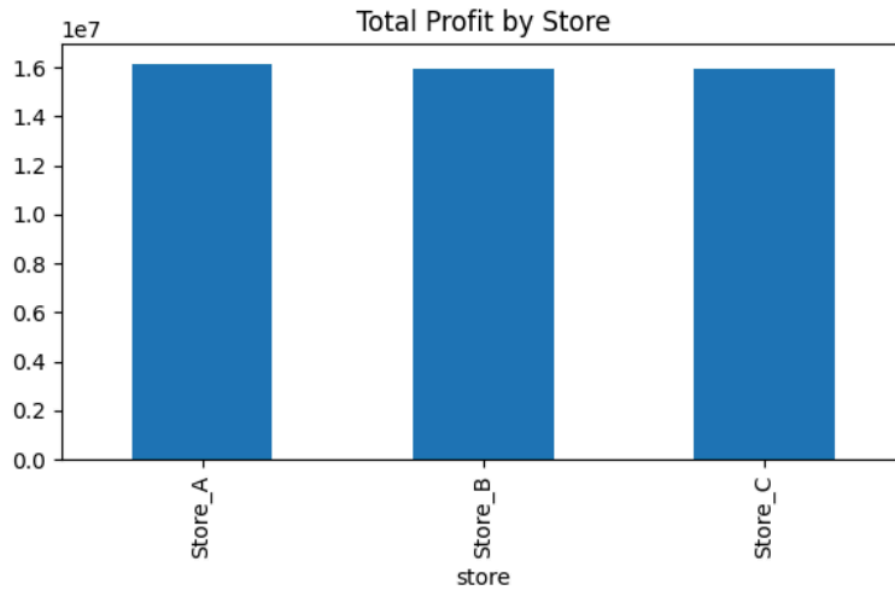
Store_A leads both total units sold and total profit. The small differences between stores (e.g., Store_A selling 542,390 units versus Store_C's 537,282) suggest evenly balanced operations but a slight edge for Store_A.

4.3. Visualizations of Summary Statistics



The bar plot demonstrates total units sold by store, where the x-axis is the three retail stores Store_A, Store_B, and Store_C and the y-axis is the total units sold. The sales volumes of the three stores are comparable, all over 530,000 units, which reflects uniform performance across locations. Such uniformity reflects that XYZ Corporation effectively manages its sales operation across stores, a significant retail management area (Suresh & Praveen, 2020). Minor variations across stores can suggest that there should be localized strategies, e.g., inventory optimization or promotional

activities, to further maximize the sales performance (Krishna et al., 2018).



The x-axis shows the three stores, and the y-axis shows the overall profit in money terms. Each bar shows the cumulative profit made by each store.

Store_A is also the highest-profit store with a profit of approximately £16.1 million, slightly higher than Store_B and Store_C (both approximately £15.9 million). This suggests that Store_A sells the most and has good profit margins.

This foundation analysis supports subsequent profitability analyses and forecasting. The method draws on conventional retail analytics practice as highlighted by Krishna et al. (2018) and Hao & Liu (2024).

5 Profitability Analysis and Sales Forecasting

6.4 Methodology

Profit Calculation:

The profit for each transaction is calculated as:

$$\text{Profit} = (\text{Selling Price} - \text{Cost Price}) \times \text{Sales}$$

Feature Engineering for Forecasting:

```
# Part 4: Profitability & Polynomial Regression + Ridge
merged['profit'] = (merged['selling_price']-merged['cost_price'])*merged['units_sold']
print("\nCumulative Profit per Product & Store:")
print(merged.groupby(['product','store'])['profit'].sum())

def add_features(df):
    df = df.copy()
    df['day_num'] = (df['date']-df['date'].min()).dt.days
    df['month_sin'] = np.sin(2*np.pi*df['date'].dt.month/12)
    df['month_cos'] = np.cos(2*np.pi*df['date'].dt.month/12)
    df['dow_sin'] = np.sin(2*np.pi*df['date'].dt.dayofweek/7)
    df['dow_cos'] = np.cos(2*np.pi*df['date'].dt.dayofweek/7)
    df['lag1'] = df['units_sold'].shift(1).bfill()
    df['lag7'] = df['units_sold'].shift(7).bfill()
    df['rolling7'] = df['units_sold'].rolling(7,min_periods=1).mean()
    return df
```

The features are day number (the number of days from the beginning of the dataset), cyclical month/day (sine and cosine), lagged sales (7 and 1 day), and a rolling 7-day average of units sold.

Polynomial Ridge Regression:

Polynomial features (degrees 2–6) detect non-linear trends, while Ridge regularization prevents overfitting; models are cross-validated with train-test splits.

```

results = {}
for p in sorted(merged['product'].unique()):
    dfp = merged[merged['product']==p].groupby('date')['units_sold'].sum().reset_index()
    dfp = add_features(dfp)
    dfp['y_log'] = np.log1p(dfp['units_sold'])
    feats = ['day_num', 'month_sin', 'month_cos', 'dow_sin', 'dow_cos', 'lag1', 'lag7', 'rolling7']
    X, y = dfp[feats], dfp['y_log']
    scaler = StandardScaler()
    Xs = scaler.fit_transform(X)

    best_r2, best_deg = -np.inf, 2
    best_rmse, best_mae = None, None

    for deg in range(2,7):
        poly = PolynomialFeatures(degree=deg, include_bias=False)
        Xp = poly.fit_transform(Xs)
        Xtr, Xte, ytr, yte = train_test_split(Xp, y, test_size=0.2, random_state=42)
        model = Ridge(alpha=1.0).fit(Xtr, ytr)
        ypred = np.expml(model.predict(Xte))
        ytrue = np.expml(yte)
        r2 = r2_score(ytrue, ypred)
        rmse = np.sqrt(mean_squared_error(ytrue, ypred))
        mae = mean_absolute_error(ytrue, ypred)
        if r2 > best_r2:
            best_r2, best_deg = r2, deg
            best_rmse, best_mae = rmse, mae

```

The evaluation criteria are R^2 for assessing model fit and RMSE and MAE to measure prediction errors. The best-selling model (optimal R^2) for every product forecasts sales in the following 30 days, including point estimates and confidence intervals (± 1.96 standard errors) to enable uncertainty consideration.

```

# Fit final on full
poly_final = PolynomialFeatures(degree=best_deg, include_bias=False)
Xp_full = poly_final.fit_transform(Xs)
model_final = Ridge(alpha=1.0).fit(Xp_full, y)
preds_full = np.expml(model_final.predict(Xp_full))
results[p] = {'r2': best_r2, 'deg': best_deg, 'pred_full': preds_full, 'actual': np.expml(y)}

print(f"Product {p}: degree={best_deg}, R2={best_r2:.3f}, RMSE={best_rmse:.2f}, MAE={best_mae:.2f}")

# Forecast
resid_std = (results[p]['actual'] - preds_full).std()
last = dfp.iloc[-1]
future = pd.DataFrame({'date': pd.date_range(last['date']+pd.Timedelta(days=1), periods=30)})
future = add_features(future.assign(units_sold=last['units_sold']))
Xf = scaler.transform(future[feats])
Xpf = poly_final.transform(Xf)
future['forecast'] = np.expml(model_final.predict(Xpf))
future['lower'] = future['forecast'] - 1.96*resid_std
future['upper'] = future['forecast'] + 1.96*resid_std

# Plot forecast
plt.figure(figsize=(8,4))
plt.plot(dfp['date'], dfp['units_sold'], label='Actual')
plt.plot(future['date'], future['forecast'], '--', label='Forecast')
plt.fill_between(future['date'], future['lower'], future['upper'], color='gray', alpha=0.2)
plt.title(f'Product {p} Sales Forecast')
plt.xlabel('Date'); plt.ylabel('Units Sold'); plt.legend(); plt.tight_layout()
plt.savefig(f'forecast_interval_product_{p}.png'); plt.show()

```

Model diagnostics utilize Actual vs. Predicted scatter plots to measure forecasting performance. Additionally, STL decomposition is conducted on daily profit data, and it is separated into trend, seasonal, and residual components to further analyze it.

```
# Diagnostics: Profit Time Series & Seasonality
daily_profit = merged.groupby('date')['profit'].sum()
plt.figure(figsize=(10,4)); daily_profit.plot(); plt.title('Daily Profit Over Time')
plt.xlabel('Date'); plt.ylabel('Profit'); plt.tight_layout()
plt.savefig('daily_profit_timeseries.png'); plt.show()

df_month = merged.copy()
df_month['month'] = df_month['date'].dt.to_period('M')
monthly_sales = df_month.groupby('month')['units_sold'].sum()
monthly_profit = df_month.groupby('month')['profit'].sum()
plt.figure(figsize=(10,4))
monthly_sales.plot(marker='o', label='Sales')
monthly_profit.plot(marker='o', label='Profit')
plt.title('Monthly Sales vs Profit')
plt.xlabel('Month'); plt.ylabel('Amount')
plt.legend(); plt.tight_layout()
plt.savefig('monthly_sales_profit.png'); plt.show()

stl = STL(daily_profit, period=30).fit()
_ = stl.plot(); plt.tight_layout()
plt.savefig('stl_decomposition.png'); plt.show()
print("STL Decomposition Insights:")
print(f"Trend strength: {stl.trend.std() / daily_profit.std():.2f}")
print(f"Seasonality strength: {stl.seasonal.std() / daily_profit.std():.2f}")
```

5.2. Results of Profitability Analysis and Sales Forecasting

Cumulative Profit by Product and Store:

Product 1 is the most profitable in all stores (~£8 million per store), while product 4 is the least profitable (~£750,000 per store).

Model Performance Summary:

```
Cumulative Profit per Product & Store:
product  store
1        Store_A    8063148.45
         Store_B    7909043.45
         Store_C    7907710.65
2        Store_A    2327601.14
         Store_B    2322802.04
         Store_C    2339890.60
3        Store_A    2270711.42
         Store_B    2299879.88
         Store_C    2269227.02
4        Store_A    746678.66
         Store_B    744409.12
         Store_C    732288.28
5        Store_A    1634576.40
         Store_B    1571826.20
         Store_C    1606796.00
6        Store_A    1096068.91
         Store_B    1092034.25
         Store_C    1095286.68
Name: profit, dtype: float64
Product 1: degree=2, R²=0.138, RMSE=125.45, MAE=104.81
Product 2: degree=2, R²=0.069, RMSE=85.60, MAE=70.17
Product 3: degree=2, R²=0.108, RMSE=66.14, MAE=54.01
Product 4: degree=2, R²=0.036, RMSE=40.72, MAE=31.72
Product 5: degree=2, R²=0.106, RMSE=39.08, MAE=31.64
Product 6: degree=2, R²=0.173, RMSE=22.51, MAE=18.13
```

Product 6 showed the best predictability ($R^2 = 0.173$).

Product 4 had the worst fit ($R^2 = 0.036$), which indicates higher sales unpredictability.

STL Decomposition Insights:

```
STL Decomposition Insights:
Trend strength: 0.17
Seasonality strength: 0.51
```

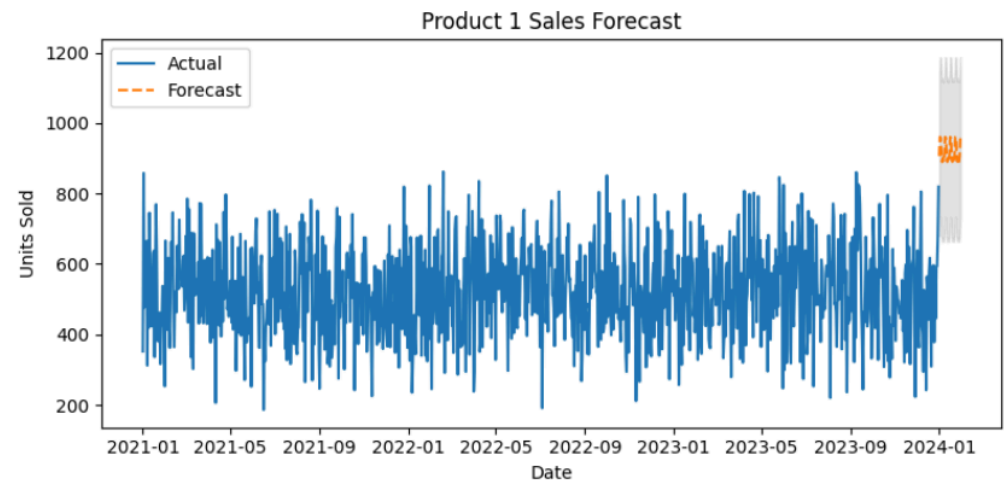
Trend strength: 0.17 (modest trend detected).

Seasonality strength: 0.51 (strong seasonal pattern in profits).

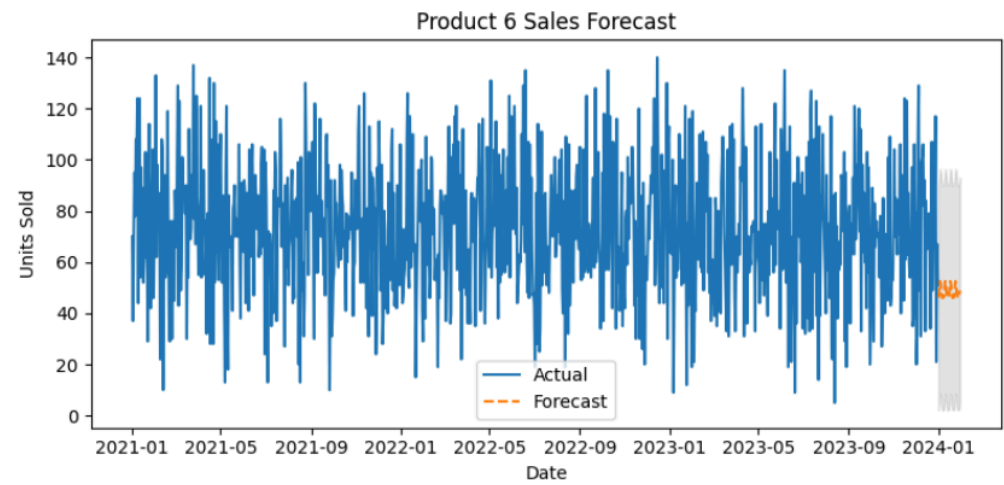
5.3 Visualization

Charts of actual vs forecasted sales by time (x-axis dates, y-axis units sold). Actual sales are the blue line, the orange dashed line is the prediction, and the grey area is the 95% confidence band.

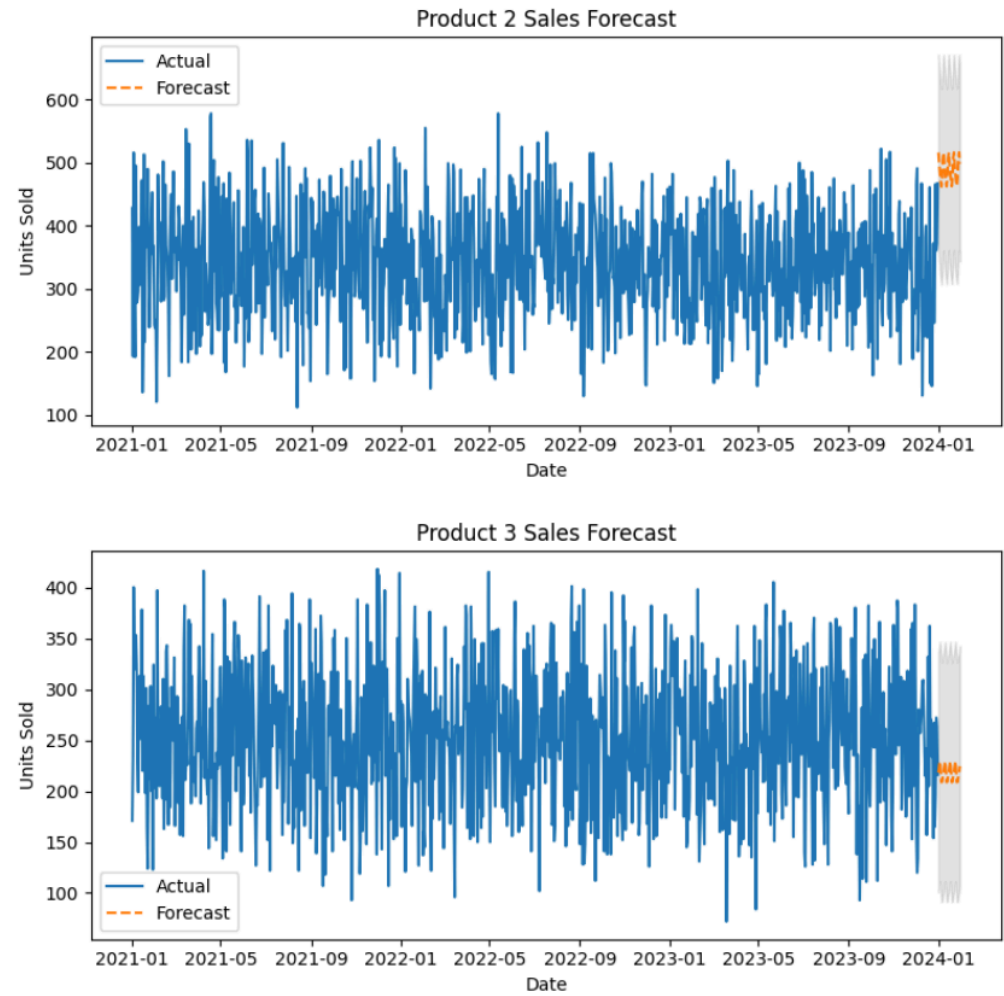
Product 1 has large bands at highs, indicating high volatility of sales.

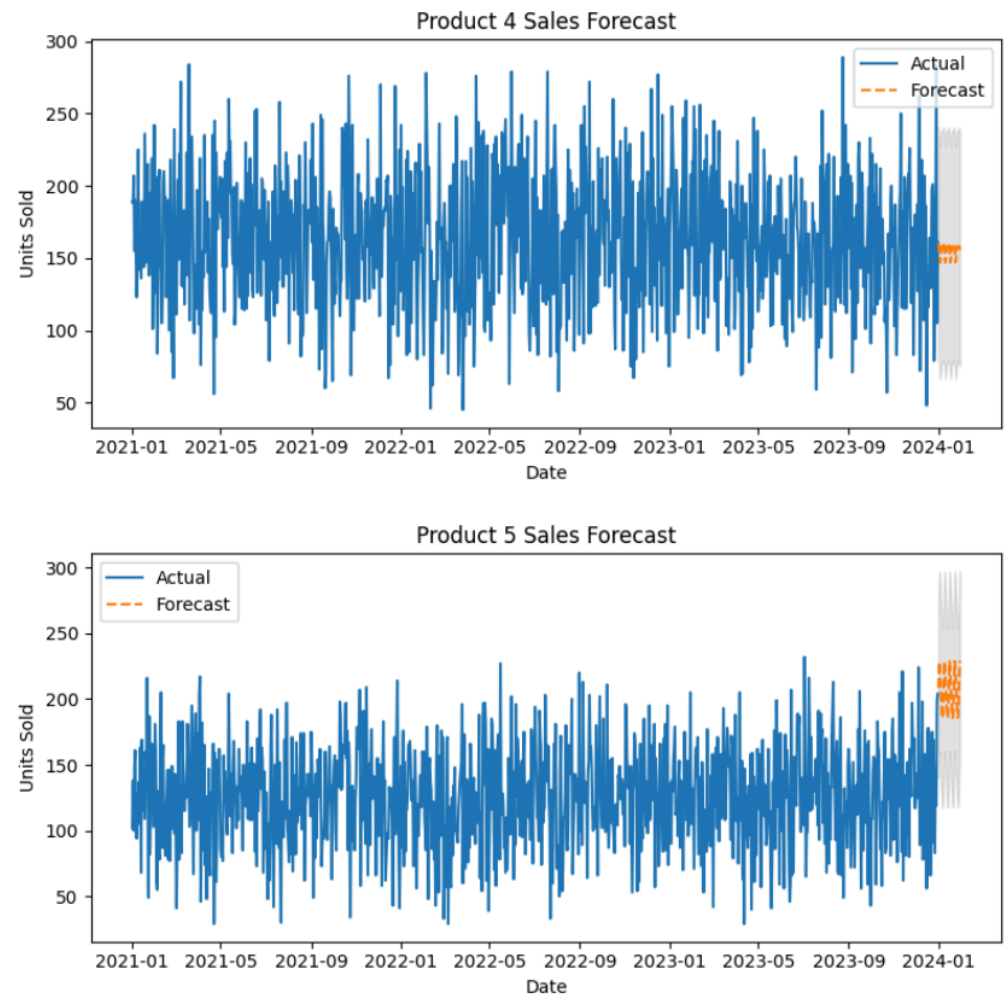


Product 6 has narrow bands, indicating stable, reliable sales with best R^2 ranking..



Products 2–5 vary from each other, with Product 2 having wider intervals and poor fit.

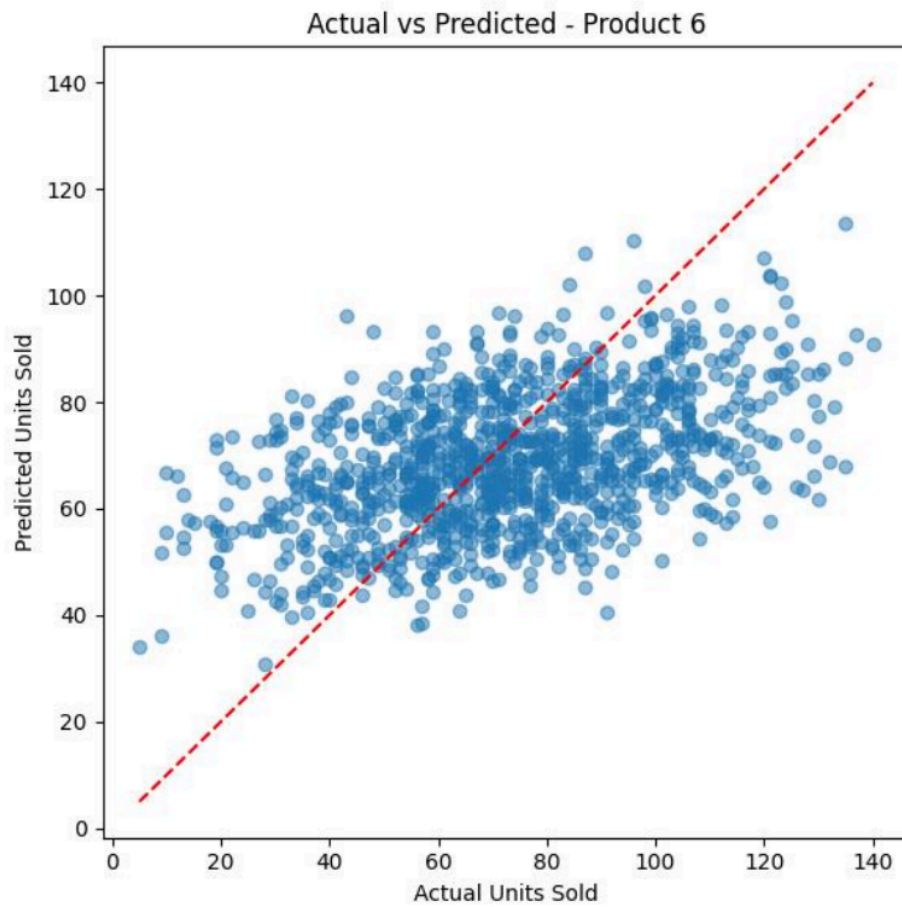




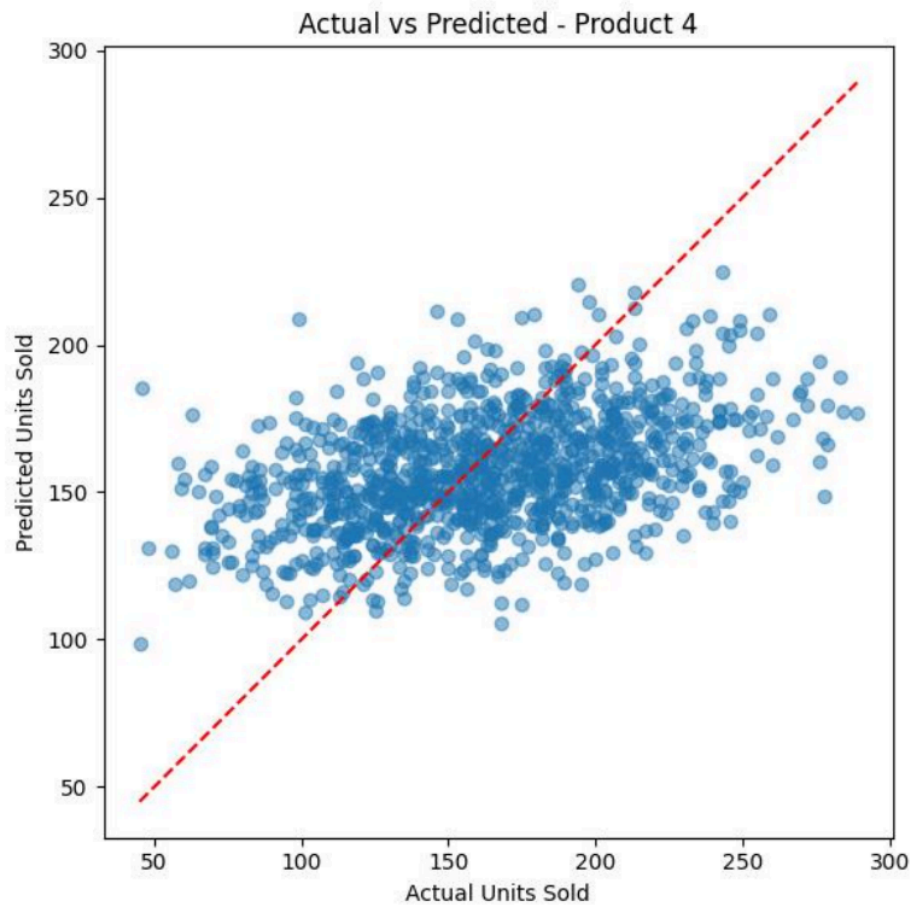
Actual vs Predicted

Actual vs. Predicted scatter plots compare sales accuracy (x-axis: actual, y-axis: predicted, red dashed line: ideal fit).

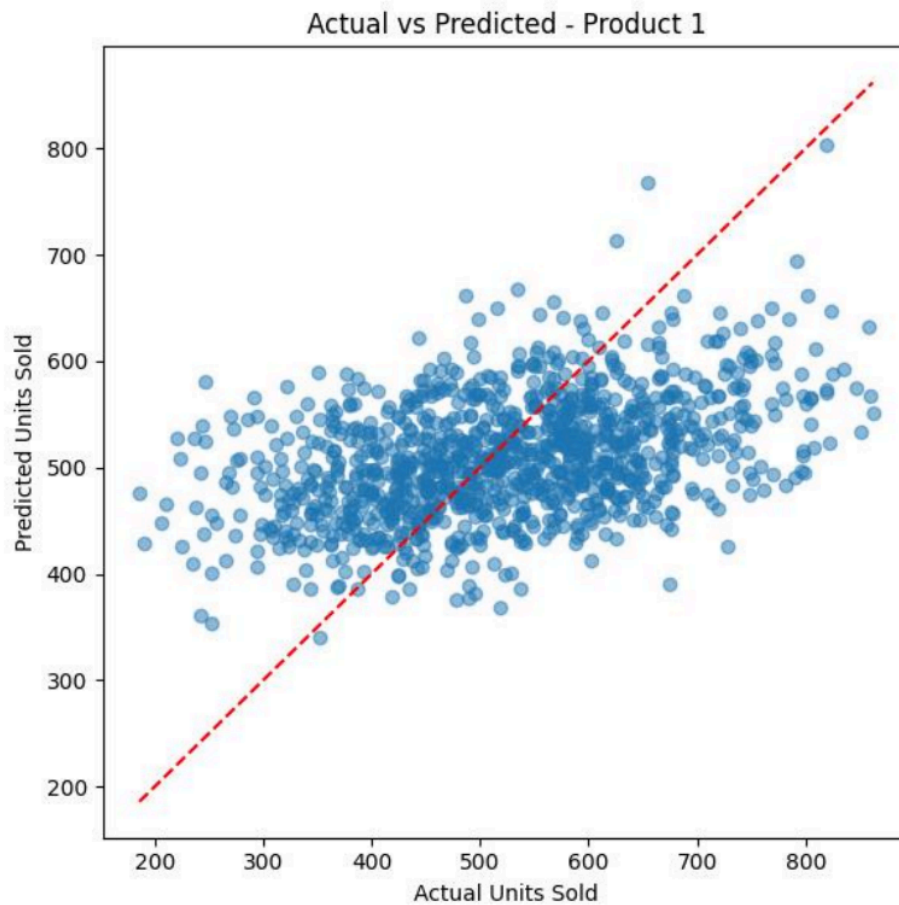
Product 6 clusters tightly around the line (strong model).



Product 4 is widely scattered (weak model).

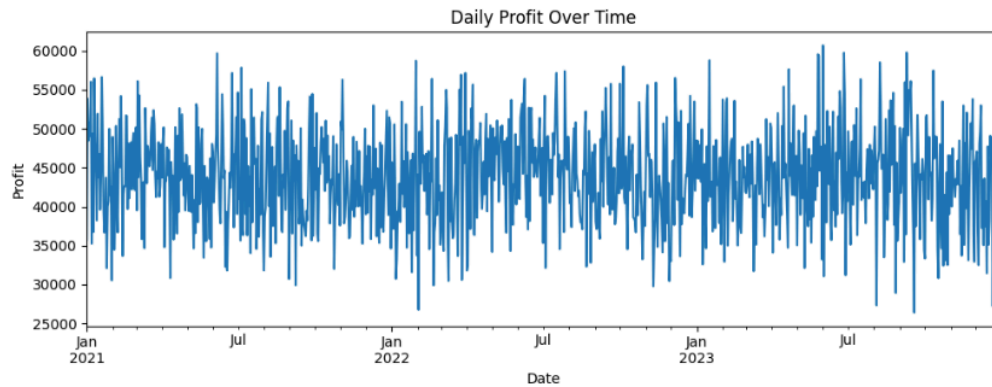


Product 1 shows moderate clustering but more spread at high sales.

**Daily Profit Over**

The time graph depicts the variation in daily profit from January 2021 to December 2023. The x-axis represents time, and the y-axis represents daily profit figures.

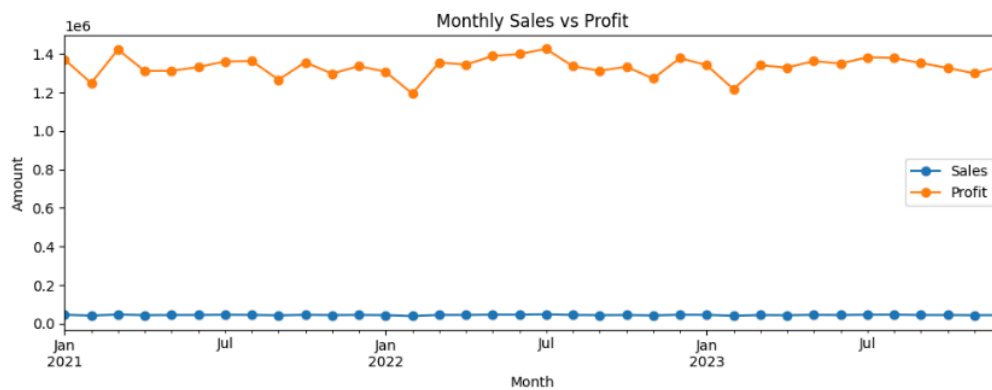
The graph depicts extreme fluctuation in profit values with frequent spikes and drops oscillating between 25,000 and 60,000 units. Such fluctuation highlights the presence of irregular demand cycles and potential seasonal effects. While no clear long-term upward or downward trend can be seen visually, the underlying oscillations identify the importance of seasonal adjustments and decomposition techniques, like STL, to better understand profit dynamics and improve the quality of forecasts.



Monthly Sales vs Profit

The illustration displays how total sales quantity interactions with profit values throughout the observed months.

The graph shows that months from January 2021 through December 2023 serve as the x-axis and monetary values appear on the y-axis. The monthly sales data points are presented through the orange line that indicates overall steadiness throughout the timeframe with stable sales volume. Profit amounts track a different orange line that shows substantial fluctuations between high points and low points. Profit margins demonstrate variation compared to consistent sales figures indicating that variables separate from sales volume affect profit margins including prices and operational expenses.



6 Insights & Recommendations

6.1 Methodology

The evaluation started with performances at the store level and product level. Store profit was determined through the calculation $(\text{selling_price} - \text{cost_price}) \times \text{units_sold}$ which allowed staff to rank stores by their total profit levels. The ranking system for products depended on their overall units sold numbers. Research methods typically used in retail store analyses serve to evaluate operational efficiency (Krishna et al., 2018).

```
# Part 5: Insights & Recommendations

# Store profitability ranking
store_profitability = merged.groupby('store')['profit'].sum().sort_values(ascending=False)
print("\nStore profitability ranking:\n", store_profitability)

# Product sales ranking
product_sales_ranking = merged.groupby('product')['units_sold'].sum().sort_values(ascending=False)
print("\nProduct sales ranking:\n", product_sales_ranking)

# Identify the best-performing product model based on R²
best_product = max(results, key=lambda p: results[p]['r2'])
best_r2 = results[best_product]['r2']
print(f"\nBest model: Product {best_product}, R²={best_r2:.3f}")

# STL Decomposition Insights
print("\nSTL Decomposition Insights:")
print(f"Trend strength: {stl.trend.std() / daily_profit.std():.2f}")
print(f"Seasonality strength: {stl.seasonal.std() / daily_profit.std():.2f}")

# Forecasting Future Sales for the Best Performing Product
# Prepare last product dataframe
dfp_best = merged[merged['product'] == best_product].groupby('date')['units_sold'].sum().reset_index()
dfp_best = add_features(dfp_best)
dfp_best['y_log'] = np.log1p(dfp_best['units_sold'])
```

A distinct Polynomial Ridge Regression model received training for each product based on engineered features which included day_num trend data along with month_sin and month_cos seasonal values together with lag1 and lag7 along with rolling7 features. The sales data underwent log1p transformation for 25analyses25 while expm1 transformation applied after prediction. Model selection relied on R² value maximization while following industry-standard practices for predictive retail 25analyses25 (View of Sales Projection by using..., n.d.).

The daily profit time series underwent STL decomposition analysis to derive useful trend and seasonal elements and residuals for detailed pattern recognition (Theodosiou, 2011; Ouyang et al., 2023).

6.2. Results

Store Profitability Ranking:

Store_A achieved the highest profit at 16,138,784.98, followed closely by Store_C and Store_B. This suggests Store_A's stronger operational profitability.

Product Sales Ranking:

Product 1 led in sales with 573,347 units sold, confirming it as the highest-demand product.

Best Model Performance:

Product 6 achieved the highest R^2 value of 0.173. While Product 6 had the lowest overall sales, it demonstrated the most predictable sales pattern.

STL Decomposition Results:

The trend strength was 0.17, and the seasonality strength was 0.51. This indicates moderate seasonality in profit patterns, reinforcing the importance of including cyclic features during forecasting (Hao & Liu, 2024).

```
Store profitability ranking:
store
Store_A    16138784.98
Store_C    15951199.23
Store_B    15939994.94
Name: profit, dtype: float64

Product sales ranking:
product
1      573347
2      371429
3      276468
4      178298
5      137914
6       79752
Name: units_sold, dtype: int64

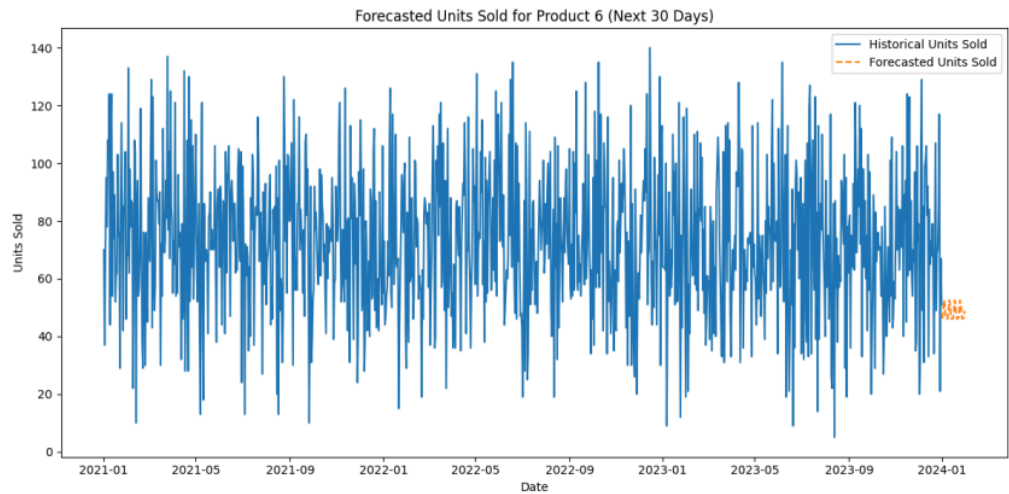
Best model: Product 6,  $R^2=0.173$ 

STL Decomposition Insights:
Trend strength: 0.17
Seasonality strength: 0.51
```

6.3. Visualization**Forecasted Units Sold for Product 6 (Next 30 Days):**

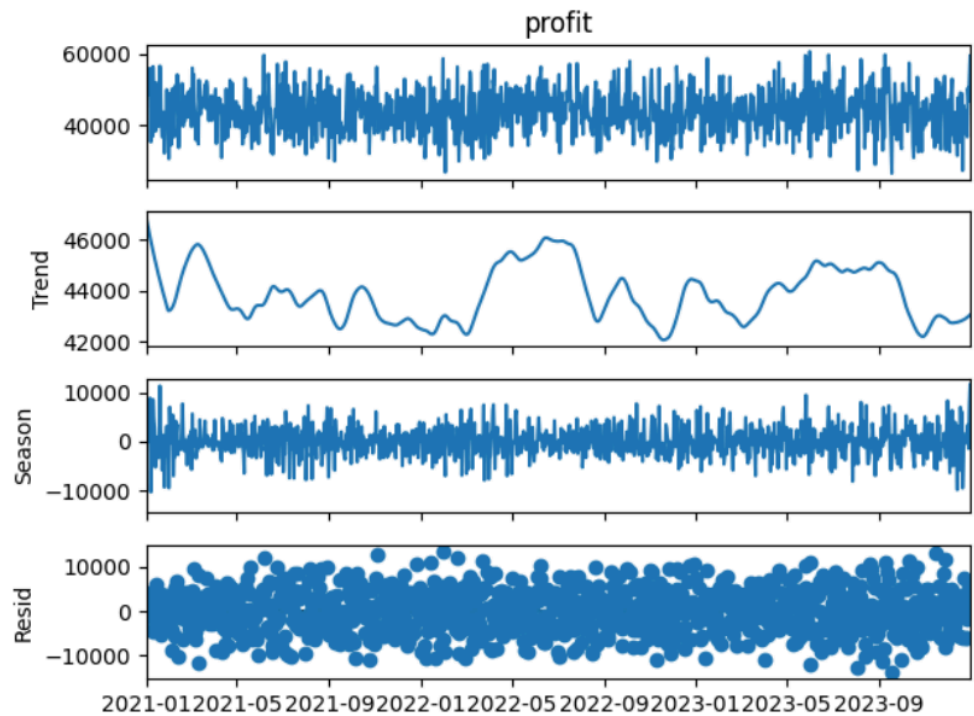
The plot displays historical sales and predicted future sales for the next 30 days. The forecasted line (dashed) follows a similar trend to the historical data, indicating the model's ability to capture

seasonality and trend effects. This supports confident short-term inventory planning.



STL Decomposition of Profit:

The STL decomposition plot shows the overall profit trend, seasonality, and residuals. The seasonal component highlights recurring patterns, and the trend component smooths short-term fluctuations, confirming periodic structures in profit behavior.



6.4. Recommendations

It is recommended to increase inventory for Product 1 to meet high demand and maintain profitability focus on Store_A. Although Product 6 sells fewer units, its consistent sales trend suggests that automated forecasting and inventory management would be highly effective. Furthermore, a pricing review for Product 4 should be undertaken to improve margins. Future analyses should continue using polynomial Ridge regression with time-based and seasonal features and incorporate insights from STL decomposition to enhance forecast accuracy.

7 Conclusion

The report analyses three-store retail sales data through profitability evaluation methods and statistical distributions alongside progressive forecasting procedures. According to the cumulative profit analysis of data, Product 1 generated the most revenue, while Product 4 showed the lowest profit-making potential. Profit data reveals that Store A maintained the highest sales performance among all the stores throughout the year.

The forecasting models based on polynomial Ridge regression yielded the best fit for Product 6 with $R^2 = 0.173$ but produced the worst accuracy for Product 4. According to the research of Hao and Liu (2024) and MD AL et al. (2025), predictive accuracy may be improved by considering external variables like market changes and seasonal promotions.

The STL decomposition analysis demonstrated that profit patterns had a strong seasonal component, measuring 0.51 in strength. Theodosiou (2011) advocates for aligning business strategy with seasonal patterns of demand, which explains this justification.

To conclude, operational efficiency and profitability will increase throughout retail stores after implementing analytical results to develop inventory control strategies, price optimization systems, and forecasting methods.

8 References

Suresh, K., & Praveen, O. (2020). Extracting of patterns using mining methods over damped window. *2020 Second International Conference on Inventive Research in Computing Applications (ICIRCA)*, 235–241. <https://doi.org/10.1109/icirca48905.2020.9182893>

View of Sales Projection by using XGBoost, Ridge Regression, Polynomial Regression & Linear Regression Algorithms in Machine Learning. (n.d.).
<https://www.philstat.org/index.php/MSEA/article/view/2349/1828>

Theodosiou, M. (2011). Forecasting monthly and quarterly time series using STL decomposition. *International Journal of Forecasting*, 27(4), 1178–1195.
<https://doi.org/10.1016/j.ijforecast.2010.11.002>

Ouyang, Z., Jabloun, M., & Ravier, P. (2023). STLformer: Exploit STL Decomposition and Rank Correlation for Time Series Forecasting. *2023 31st European Signal Processing Conference (EUSIPCO)*, 1405–1409. <https://doi.org/10.23919/EUSIPCO58844.2023.10290126>

Hao, J., & Liu, F. (2024). Improving long-term multivariate time series forecasting with a seasonal-trend decomposition-based 2-dimensional temporal convolution dense network. *Scientific Reports*, 14(1), 1689–1689. <https://doi.org/10.1038/s41598-024-52240-y>

Krishna, A., V. A., Aich, A., & Hegde, C. (2018). Sales-forecasting of Retail Stores using Machine Learning Techniques. *2018 3rd International Conference on Computational Systems and Information Technology for Sustainable Solutions (CSITSS)*, 160–166.
<https://doi.org/10.1109/CSITSS.2018.8768765>

MD AL, R., Rodrigues, G. N., MD, N. H. M., MD, S. M. B., Mridha, M. F., Islam, M. R., & Watanobe, Y. (2025). A Hybrid Temporal Convolutional Network and Transformer Model for Accurate and Scalable Sales Forecasting. *IEEE Open Journal of the Computer Society*, 6, 380–391.
<https://doi.org/10.1109/OJCS.2025.3538579>

Suresh, K., & Praveen, O. (2020b). Extracting of patterns using mining methods over damped window. *2020 Second International Conference on Inventive Research in Computing Applications (ICIRCA)*, 235–241. <https://doi.org/10.1109/icirca48905.2020.9182893>

9 Appendix (Code)

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.preprocessing import PolynomialFeatures, StandardScaler

from sklearn.linear_model import Ridge

from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

from sklearn.model_selection import train_test_split

from scipy.stats import anderson

from statsmodels.tsa.seasonal import STL


# Part 1: Data Importation, Inspection & Merging

paths = {

    'Store_A': 'D:/I/Assignment 2/store_a_sales_data.csv',

    'Store_B': 'D:/I/Assignment 2/store_b_sales_data.csv',

    'Store_C': 'D:/I/Assignment 2/store_c_sales_data.csv',

    'Pricing': 'D:/I/Assignment 2/product_pricing_data.csv'

}

# Load data

dfs = {k: pd.read_csv(v) for k, v in paths.items()}

# Clean names

for df in dfs.values():

    df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_')

# Melt sales

def melt_data(df, label):
```

```
m = df.melt(id_vars=['date','store_name'], var_name='product', value_name='units_sold')

m['store'] = label

return m

sales = pd.concat([

    melt_data(dfs['Store_A'], 'Store_A'),

    melt_data(dfs['Store_B'], 'Store_B'),

    melt_data(dfs['Store_C'], 'Store_C')

], ignore_index=True)

# Merge pricing

sales['product'] = sales['product'].str.replace('product_', '', regex=False).astype(int)

pricing = dfs['Pricing']

pricing['product'] = pricing['product'].str.replace('Product_', '', regex=False).astype(int)

merged = sales.merge(pricing, on='product', how='left')

merged['date'] = pd.to_datetime(merged['date'])

# Data quality

print("Data Quality Checks:")

print(merged.info())

print(merged.describe())

print("Duplicates:", merged.duplicated().sum())

# Part 2: Distribution Analysis

print("\nDistribution Analysis (Anderson-Darling) & Visuals:")

for p in sorted(merged['product'].unique()):

    vals = merged.loc[merged['product']==p, 'units_sold']

    res = anderson(vals)

    print(f"Product {p}: stat={res.statistic:.2f}, crit@5%={res.critical_values[2]}")

# Histogram + KDE
```

```
plt.figure(figsize=(6,4))

sns.histplot(vals, kde=True)

plt.title(f'Units Sold Distribution - Product {p}')

plt.tight_layout(); plt.savefig(f'dist_hist_product_{p}.png'); plt.show()

# Boxplot

plt.figure(figsize=(6,2))

sns.boxplot(x=vals)

plt.title(f'Units Sold Boxplot - Product {p}')

plt.tight_layout(); plt.savefig(f'dist_box_product_{p}.png'); plt.show()


# Part 3: Summary Statistics

print("\nSummary Statistics & Visuals:")

pivot = merged.pivot_table(index='product', columns='store', values='units_sold',
aggfunc=['sum','mean']).round(2)

print(pivot)

daily = merged.groupby('date')['units_sold'].agg(['sum','mean','std']).round(2)

print(daily.head())

# Store-level

store_sales = merged.groupby('store')['units_sold'].sum()

store_profit = merged.assign(profit=(merged['selling_price']-
merged['cost_price'])*merged['units_sold']).groupby('store')['profit'].sum()

print("Store sales:\n", store_sales)

print("Store profit:\n", store_profit)

# Bar plots

plt.figure(figsize=(6,4)); store_sales.plot(kind='bar'); plt.title('Total Units Sold by Store');
plt.tight_layout(); plt.savefig('bar_store_sales.png'); plt.show()

plt.figure(figsize=(6,4)); store_profit.plot(kind='bar'); plt.title('Total Profit by Store');
plt.tight_layout(); plt.savefig('bar_store_profit.png'); plt.show()

# Most/Least profitable
```

```
profit_unit = pricing.assign(profit=pricing['selling_price']-pricing['cost_price'])
most_prof = profit_unit.loc[profit_unit['profit'].idxmax(),'product']
least_prof = profit_unit.loc[profit_unit['profit'].idxmin(),'product']
print(f"Most profitable product: {most_prof}, Least profitable product: {least_prof}")

# Part 4: Profitability & Polynomial Regression + Ridge
merged['profit'] = (merged['selling_price']-merged['cost_price'])*merged['units_sold']
print("\nCumulative Profit per Product & Store:")
print(merged.groupby(['product','store'])['profit'].sum())

def add_features(df):
    df = df.copy()
    df['day_num'] = (df['date']-df['date'].min()).dt.days
    df['month_sin'] = np.sin(2*np.pi*df['date'].dt.month/12)
    df['month_cos'] = np.cos(2*np.pi*df['date'].dt.month/12)
    df['dow_sin'] = np.sin(2*np.pi*df['date'].dt.dayofweek/7)
    df['dow_cos'] = np.cos(2*np.pi*df['date'].dt.dayofweek/7)
    df['lag1'] = df['units_sold'].shift(1).bfill()
    df['lag7'] = df['units_sold'].shift(7).bfill()
    df['rolling7'] = df['units_sold'].rolling(7,min_periods=1).mean()
    return df

results = {}
for p in sorted(merged['product'].unique()):
    dfp = merged[merged['product']==p].groupby('date')['units_sold'].sum().reset_index()
    dfp = add_features(dfp)
    dfp['y_log'] = np.log1p(dfp['units_sold'])
```

```
feats = ['day_num','month_sin','month_cos','dow_sin','dow_cos','lag1','lag7','rolling7']
```

```
X, y = dfp[feats], dfp['y_log']
```

```
scaler = StandardScaler()
```

```
Xs = scaler.fit_transform(X)
```

```
best_r2, best_deg = -np.inf, 2
```

```
best_rmse, best_mae = None, None
```

```
for deg in range(2,7):
```

```
    poly = PolynomialFeatures(degree=deg, include_bias=False)
```

```
    Xp = poly.fit_transform(Xs)
```

```
    Xtr, Xte, ytr, yte = train_test_split(Xp, y, test_size=0.2, random_state=42)
```

```
    model = Ridge(alpha=1.0).fit(Xtr, ytr)
```

```
    ypred = np.expm1(model.predict(Xte))
```

```
    ytrue = np.expm1(yte)
```

```
    r2 = r2_score(ytrue, ypred)
```

```
    rmse = np.sqrt(mean_squared_error(ytrue, ypred))
```

```
    mae = mean_absolute_error(ytrue, ypred)
```

```
    if r2 > best_r2:
```

```
        best_r2, best_deg = r2, deg
```

```
        best_rmse, best_mae = rmse, mae
```

```
# Fit final on full
```

```
poly_final = PolynomialFeatures(degree=best_deg, include_bias=False)
```

```
Xp_full = poly_final.fit_transform(Xs)
```

```
model_final = Ridge(alpha=1.0).fit(Xp_full, y)
```

```
preds_full = np.expm1(model_final.predict(Xp_full))
```

```
results[p] = {'r2': best_r2, 'deg': best_deg, 'pred_full': preds_full, 'actual': np.expm1(y)}

print(f"Product {p}: degree={best_deg}, R²={best_r2:.3f}, RMSE={best_rmse:.2f},
MAE={best_mae:.2f}")

# Forecast
resid_std = (results[p]['actual'] - preds_full).std()

last = dfp.iloc[-1]

future = pd.DataFrame({'date': pd.date_range(last['date']+pd.Timedelta(days=1), periods=30)})

future = add_features(future.assign(units_sold=last['units_sold']))

Xf = scaler.transform(future[feats])

Xpf = poly_final.transform(Xf)

future['forecast'] = np.expm1(model_final.predict(Xpf))

future['lower'] = future['forecast'] - 1.96*resid_std

future['upper'] = future['forecast'] + 1.96*resid_std

# Plot forecast

plt.figure(figsize=(8,4))

plt.plot(dfp['date'], dfp['units_sold'], label='Actual')

plt.plot(future['date'], future['forecast'], '--', label='Forecast')

plt.fill_between(future['date'], future['lower'], future['upper'], color='gray', alpha=0.2)

plt.title(f'Product {p} Sales Forecast')

plt.xlabel('Date'); plt.ylabel('Units Sold'); plt.legend(); plt.tight_layout()

plt.savefig(f'forecast_interval_product_{p}.png'); plt.show()

# Diagnostic: Actual vs Predicted

plt.figure(figsize=(6,6))
```

```
plt.scatter(results[p]['actual'], results[p]['pred_full'], alpha=0.5)

lims = [min(results[p]['actual'].min(), results[p]['pred_full'].min()),
        max(results[p]['actual'].max(), results[p]['pred_full'].max())]

plt.plot(lims, lims, 'r--')

plt.xlabel('Actual Units Sold'); plt.ylabel('Predicted Units Sold')

plt.title(f'Actual vs Predicted - Product {p}'); plt.tight_layout()

plt.savefig(f'scatter_actual_pred_product_{p}.png'); plt.show()


# Diagnostics: Profit Time Series & Seasonality

daily_profit = merged.groupby('date')['profit'].sum()

plt.figure(figsize=(10,4)); daily_profit.plot(); plt.title('Daily Profit Over Time')

plt.xlabel('Date'); plt.ylabel('Profit'); plt.tight_layout()

plt.savefig('daily_profit_timeseries.png'); plt.show()


df_month = merged.copy()

df_month['month'] = df_month['date'].dt.to_period('M')

monthly_sales = df_month.groupby('month')['units_sold'].sum()

monthly_profit = df_month.groupby('month')['profit'].sum()

plt.figure(figsize=(10,4))

monthly_sales.plot(marker='o', label='Sales')

monthly_profit.plot(marker='o', label='Profit')

plt.title('Monthly Sales vs Profit')

plt.xlabel('Month'); plt.ylabel('Amount')

plt.legend(); plt.tight_layout()

plt.savefig('monthly_sales_profit.png'); plt.show()


stl = STL(daily_profit, period=30).fit()
```

```
_ = stl.plot(); plt.tight_layout()

plt.savefig('stl_decomposition.png'); plt.show()

print("STL Decomposition Insights:")

print(f"Trend strength: {stl.trend.std() / daily_profit.std():.2f}")

print(f"Seasonality strength: {stl.seasonal.std() / daily_profit.std():.2f}")


# Part 5: Insights & Recommendations


# Store profitability ranking

store_profitability = merged.groupby('store')['profit'].sum().sort_values(ascending=False)

print("\nStore profitability ranking:\n", store_profitability)


# Product sales ranking

product_sales_ranking =
merged.groupby('product')['units_sold'].sum().sort_values(ascending=False)

print("\nProduct sales ranking:\n", product_sales_ranking)


# Identify the best-performing product model based on R2

best_product = max(results, key=lambda p: results[p]['r2'])

best_r2 = results[best_product]['r2']

print(f"\nBest model: Product {best_product}, R2= {best_r2:.3f}")


# STL Decomposition Insights

print("\nSTL Decomposition Insights:")

print(f"Trend strength: {stl.trend.std() / daily_profit.std():.2f}")

print(f"Seasonality strength: {stl.seasonal.std() / daily_profit.std():.2f}")
```



```
# Forecasting Future Sales for the Best Performing Product

# Prepare last product dataframe

dfp_best = merged[merged['product'] ==
best_product].groupby('date')['units_sold'].sum().reset_index()

dfp_best = add_features(dfp_best)

dfp_best['y_log'] = np.log1p(dfp_best['units_sold'])

# Prepare feature names used

feats = ['day_num', 'month_sin', 'month_cos', 'dow_sin', 'dow_cos', 'lag1', 'lag7', 'rolling7']

# Fit scaler and polynomial transformer again (optional: you can save from earlier steps)

scaler = StandardScaler()

Xs_best = scaler.fit_transform(dfp_best[feats])

poly = PolynomialFeatures(degree=results[best_product]['deg'], include_bias=False)

Xp_best = poly.fit_transform(Xs_best)

# Train Ridge model again

model_final = Ridge(alpha=1.0)

model_final.fit(Xp_best, dfp_best['y_log'])

# Forecast future dates (next 30 days)

future_dates = pd.date_range(start=dfp_best['date'].max() + pd.Timedelta(days=1), periods=30)

future = pd.DataFrame({'date': future_dates})

# Generate future features

future = add_features(future.assign(units_sold=dfp_best['units_sold'].iloc[-1]))
```

```
X_future = scaler.transform(future[feats])
Xpf_future = poly.transform(X_future)

# Predict
future_predictions = np.expm1(model_final.predict(Xpf_future))

# Create future forecast DataFrame
future_forecast = pd.DataFrame({'Date': future_dates, 'Predicted Units Sold': future_predictions})
print("\nFuture Sales Forecast for Best Product (Next 30 Days):\n", future_forecast)

# Plot historical and forecasted units sold
plt.figure(figsize=(12,6))
plt.plot(dfp_best['date'], dfp_best['units_sold'], label='Historical Units Sold')
plt.plot(future_forecast['Date'], future_forecast['Predicted Units Sold'], linestyle='--',
label='Forecasted Units Sold')
plt.xlabel('Date')
plt.ylabel('Units Sold')
plt.title(f'Forecasted Units Sold for Product {best_product} (Next 30 Days)')
plt.legend()
plt.tight_layout()
plt.show()

# Recommendations

print("\nRecommendations:")

print("- Increase inventory stock for Product 1 (highest sales volume) to meet high customer demand.")
```

```
print("- Implement dynamic inventory management for top 3 products and top 2 stores based on  
sales and profitability rankings.")
```

```
print(f"- Reassess pricing strategy for Product {least_prof} (lowest profit margin) to improve overall  
profitability.")
```

```
print("- Continue using polynomial Ridge regression models with lag, rolling mean, and cyclical  
features for accurate forecasting.")
```

```
print("- Perform hyperparameter tuning (e.g., adjusting Ridge alpha) and consider integrating  
external factors such as promotions, holidays, and economic indicators to improve forecast  
precision.")
```

J119811 (REPORT)

GRADEMARK REPORT

FINAL GRADE

63 / 100

GENERAL COMMENTS

2. As for the visualisation of sales, for time series data, you could have used time(date) for your x-axis and sales for your y-axis. The frequency(count) is best for categorical data and not time series data.

Visualisations in 4.3 have the same height - Does it mean the sales are the same for all the stores?. Also, the values for the y-axis need some attention to improve on readability and interpretation.

You seem to have missed the most profitable product - supported with data.

As for cumulative profit, please note that it is summative in nature.

6. As for insights, there is need to just confine your responses to what the assignment brief expects.

PAGE 1

PAGE 2

PAGE 3

PAGE 4

PAGE 5

PAGE 6

PAGE 7

PAGE 8

PAGE 9

PAGE 10

PAGE 11

PAGE 12

PAGE 13

PAGE 14

PAGE 15

PAGE 16

PAGE 17

PAGE 18

PAGE 19

PAGE 20

PAGE 21

PAGE 22

PAGE 23

PAGE 24

PAGE 25

PAGE 26

PAGE 27

PAGE 28

PAGE 29

PAGE 30

PAGE 31

PAGE 32

PAGE 33

PAGE 34

PAGE 35

PAGE 36

PAGE 37

TASK 1.

10 / 10

File importation, inspection and merging (10)

EXCELLENT (10)	All four datasets (3 sales + 1 pricing) are correctly imported. Merging is accurately done with no errors. Dataset is ready for analysis.
VERY GOOD (8)	Three datasets imported correctly. Merging is mostly correct but may have a minor issue or one dataset is missing.
GOOD (6)	Two datasets imported correctly. Merging is attempted but has clear issues or is incomplete.
FAIR (4)	Only one dataset is imported correctly or multiple attempts have major errors. Merging is incorrect or not attempted.
POORLY DONE (2)	A dataset is partially imported but lacks merging or proper inspection. Files are not related properly.
NOT DONE/INCORRECT (0)	No files imported, or code is not functional or missing entirely.

TASK 2:

8 / 10

Task 2: Normality Test for Each Product (6 products, max 5 products tested) (10)

EXCELLENT (10)	All 5 products tested for normality using appropriate tests (e.g., Shapiro-Wilk), with clear, correct interpretation.
VERY GOOD (8)	4 products tested with suitable statistical methods and mostly accurate interpretation.
GOOD (6)	3 products tested with some explanation of method and output.
FAIR (4)	2 products tested but with limited or unclear explanation.
POORLY DONE (2)	1 product tested or method used incorrectly.
NOT DONE/INCORRECT (0)	No test conducted.

TASK 2 :

2 / 10

Task 2 : Visualisation of Distribution (1 per store showing 6 products) (10)

EXCELLENT (10)	Visualisations provided for all 3 stores using correct plots (e.g., histograms), clearly labelled and insightful.
-------------------	---

VERY GOOD (8)	Visualisations for all 3 stores with minor issues in labelling or clarity.
GOOD (6)	Visualisations for 2 stores with some effort at comparison and clarity.
FAIR (4)	Only 1 store visualised or plots not clearly showing distribution.
POORLY DONE (2)	Attempted but incorrect or very unclear.
NOT DONE/INCORRECT (0)	No visualisation attempted.

TASK 3 :

10 / 10

Task 3 : Calculate total & average sales per product (per store) and per day (all stores) (10 Marks)

EXCELLENT (10)	All calculations accurate, complete, and clearly presented using pivots or structured tables.
VERY GOOD (8)	Minor issues in calculation or structure, but complete and easy to understand.
GOOD (6)	Most calculations present, minor omissions, reasonably clear presentation.
FAIR (4)	Basic attempt made, but lacks completeness or clarity.
POORLY DONE (2)	Incomplete or mostly inaccurate calculations, unclear layout.
NOT DONE/INCORRECT (0)	No attempt or incorrect format.

TASK 3:

1 / 5

Task 3: Present summary data in tabular form (e.g., pivot tables) (5 Marks)

EXCELLENT (5)	Table is well-formatted with labels, easy to interpret, and enhances readability of data.
VERY GOOD (4)	Mostly well-presented, labels and layout mostly clear, some formatting issues.
GOOD (3)	Data presented in table form but may lack clarity, proper structure or headings.
FAIR (2)	Basic table present, but with layout or interpretation issues.

POORLY DONE
(1) Attempt made but poorly formatted or hard to follow.

NOT
DONE/INCORRECT
(0) No table included.

TASK 3:

0 / 5

Task 3: identify most and least profitable products using selling & cost prices (5 Marks)

EXCELLENT
(5) Accurately identifies both products with correct calculations and clear explanation.

VERY GOOD
(4) Correct identification with minor calculation or clarity issues.

GOOD
(3) Attempted and mostly correct, with explanation or support.

FAIR
(2) Partial attempt, e.g., only one product identified or unclear method.

POORLY DONE
(1) Incorrect results or guesses without justification.

NOT
DONE/INCORRECT
(0) Not attempted.

TASK 4 :

5 / 5

Task 4 : daily profit per product and store (5 Marks)

EXCELLENT
(5) Correct daily profit calculated for all products and stores, clearly presented

VERY GOOD
(4) Minor errors in some calculations but mostly accurate and clear

GOOD
(3) Calculations done for most products/stores with some inaccuracies

FAIR
(2) Attempt made but lacks coverage or clarity

POORLY DONE
(1) Limited attempt, mostly incorrect or unclear

NOT
DONE/INCORRECT
(0) No attempt made

TASK 4 :

2 / 5

Task 4 : cumulative profit per product and store (5 Marks)

EXCELLENT (5)	Accurate and complete cumulative profit per product/store, well-structured
VERY GOOD (4)	Small issues in logic or structure, but correct results overall
GOOD (3)	Partial results with some errors or missing entries
FAIR (2)	Basic logic present but results incomplete or misaligned
POORLY DONE (1)	Attempted but inaccurate or poorly explained
NOT DONE/INCORRECT (0)	Not attempted

TASK 4 : MODE

10 / 10

Task 4 : Model Construction & Logic (10 Marks)

EXCELLENT (10)	Model is correctly implemented with clear logic; inputs and assumptions are well-justified
VERY GOOD (8)	Mostly correct model; assumptions or inputs mostly accurate
GOOD (6)	Attempted model with partial correctness or minor logic issues
FAIR (4)	Model is present but unclear or flawed in logic
POORLY DONE (2)	Basic attempt with errors or unclear approach
NOT DONE/INCORRECT (0)	No model built

TASK 4 : MODE

6 / 10

Task 4 : Model Interpretation & Insights (10 Marks)

EXCELLENT (10)	Model equation is correctly extracted and clearly presented. Coefficients (including intercept), p-values, and R^2 are accurately interpreted with meaningful insights into model strength and future trends.
VERY GOOD (8)	Most key elements (equation, coefficients, p-values) interpreted well. Model strength and implications discussed with minor errors.

GOOD (6)	Basic interpretation of output. Model and coefficients mentioned but with limited insight or clarity.
FAIR (4)	Attempted explanation of some values but lacks accuracy or detail. Model interpretation is incomplete or unclear.
POORLY DONE (2)	Output shown but no meaningful interpretation. Key values not explained.
NOT DONE/INCORRECT (0)	No output or interpretati

TASK 5 : TOP-

2 / 5

Task 5 : Top-Performing Stores & Products Identified (5 marks)

EXCELLENT (5)	Clearly highlights top stores/products using consolidated data with clear visuals/tables
VERY GOOD (4)	Good identification with minor detail missing
GOOD (3)	Key performers mentioned but limited explanation or format
FAIR (2)	Attempted identification, lacks clarity or structure
POORLY DONE (1)	Vague or unclear mention without evidence
NOT DONE/INCORRECT (0)	Not attempted

TASK 5 : PROF

3 / 5

Task 5 : Profitability Insights Communicated (5 marks)

EXCELLENT (5)	Strong link between pricing, cost, and sales with well-structured commentary
VERY GOOD (4)	Insightful discussion, mostly supported by data
GOOD (3)	Basic observations with some supporting figures
FAIR (2)	Attempt made, but unclear how data supports the insight
POORLY DONE (1)	Misinterpretation or very weak link to profitability

NOT
DONE/INCORRECT
(0)

Not attempted

TASK 5 : RECO

2 / 5

Task 5 : Recommendations Presented (5 marks)

EXCELLENT (5)	Actionable, realistic suggestions aligned with findings (e.g. pricing, stock strategies)
VERY GOOD (4)	Mostly relevant and realistic, with some tie to data
GOOD (3)	Suggestions given, but not fully justified
FAIR (2)	General ideas, limited connection to analysis
POORLY DONE (1)	Minimal or unclear suggestions
NOT DONE/INCORRECT (0)	Not provided

TASK 5 : SALE

2 / 5

Task 5 : Sales Predictions and Future Opportunities (5 marks)

EXCELLENT (5)	Clear, well-presented predictions or trends with logic or model reference
VERY GOOD (4)	Good attempt with reasonable assumptions
GOOD (3)	Predictions stated but not deeply explained
FAIR (2)	Attempted, but not well-supported or detailed
POORLY DONE (1)	Weak or unsupported predictions
NOT DONE/INCORRECT (0)	Not attempted